lecture_14

September 5, 2026

1 Lecture 14: Unconstrained Optimization — Newton's Method

`{note} Lecture 13 introduced Gradient Descent - an iterative method that follows the negative gradient without any knowledge of curvature. While easy to implement, it can converge slowly in ill-conditioned landscapes. This lecture introduces Newton's Method, which incorporates second-order (curvature) information through the Hessian to compute a much better-informed search direction.`

Learning Objectives

By the end of this notebook, you will be able to: 1. Apply first- and second-order optimality conditions to classify stationary points of a smooth unconstrained objective function as local minima, local maxima, or saddle points. 2. Trace Newton's Method step-by-step and explain why it converges faster than Gradient Descent. 3. Explain the trade-offs in convergence speed and computational cost across Gradient Descent, Newton's Method, and BFGS.

Prerequisites: NLP Principles (Lecture 12); Gradient Descent (Lecture 13); calculus (partial derivatives, Taylor expansion); basic linear algebra (matrix inverse, eigenvalues).

Estimated time: 90 minutes (including in-class exercises)

1.1 Optimality Conditions

1.1.1 First-Order Necessary Condition

If x^* is a local optimum of a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$, then the gradient:

$$\nabla f(x^*) = \mathbf{0}$$

A point satisfying $\nabla f(x^*) = \mathbf{0}$ is called a stationary point. This is a necessary condition, not sufficient — a stationary point can be a minimum, a maximum, or a saddle point.

1.1.2 Second-Order Conditions

The Hessian $\nabla^2 f(x^*)$ — the $n \times n$ matrix of second partial derivatives — tells us the curvature of f at the stationary point and resolves the ambiguity:

Hessian $\nabla^2 f(x^*)$	All eigenvalues	Character of x^*
Positive definite (PD)	> 0	Local minimum
Negative definite (ND)	< 0	Local maximum
Indefinite	Mixed signs	Saddle point

1.1.3 Example

A Jaipur last-mile delivery planner models the net daily profit of a micro-hub as a function of the number of delivery zones served x and the vehicle turnaround speed y (km/h above baseline). The profit function is:

$$P(x, y) = x^2 - y^2 + 4x - 6y + 10$$

Setting $\nabla P = \mathbf{0}$:

$$\frac{\partial P}{\partial x} = 2x + 4 = 0 \implies x^* = -2$$

$$\frac{\partial P}{\partial y} = -2y - 6 = 0 \implies y^* = -3$$

The Hessian is:

$$\nabla^2 P = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$$

The eigenvalues are $+2$ and -2 — mixed signs — so the Hessian is indefinite and $(x^*, y^*) = (-2, -3)$ is a saddle point, not an optimum. The gradient vanishes, but the planner achieves neither maximum profit nor minimum loss at this configuration. This illustrates why the first-order condition alone is insufficient: the Hessian check is essential.

1.2 Iterative Descent Framework

For a general nonlinear function f, the system of equations $\nabla f(x) = \mathbf{0}$ may be nonlinear with no closed-form solution. To this end, to find the point where the gradient vanishes for such problems, we start from an initial guess $x^{(0)}$ and repeatedly compute a direction $d^{(k)}$ that moves downhill, then take a step along it $x^{(k+1)} = x^{(k)} + \alpha^{(k)} d^{(k)}$, until the gradient is (approximately) zero. The three methods covered in Lectures 13–15 differ only in how they choose $d^{(k)}$:

Method	Direction $d^{(k)}$	Information used	Convergence
Gradient Descent	$-\nabla f(x^{(k)})$	First-order	Linear
Newton's Method	$-[\nabla^2 f(x^{(k)})]^{-1} \nabla f(x^{(k)})$	Second-order	Quadratic

Method	Direction $d^{(k)}$	Information used	Convergence
Quasi-Newton's Method (BFGS)	$-[H^{(k)}]^{-1}\nabla f(x^{(k)})$, $H^{(k)} \approx \nabla^2 f$	Accumulated first-order	Superlinear

```
{tip} In practice: solve $\nabla f(x) = \mathbf{0}$ analytically if a closed-form
solution can be established; otherwise, default to Quasi-Newton's Method; use
Newton's Method when the Hessian is cheap to compute and invert; use Gradient
Descent when problem scale makes storing a Hessian prohibitive.
```

1.3 2. Newton's Method

While Gradient Descent uses only first-order information, Newton's Method incorporates second-order (curvature) information to compute a better search direction. It fits a local quadratic model of f around $x^{(k)}$ via the Taylor expansion:

$$f(x^{(k)} + d) \approx f(x^{(k)}) + \nabla f(x^{(k)})^\top d + \frac{1}{2}\, d^\top \nabla^2 f(x^{(k)})\, d$$

Minimizing this quadratic approximation over d gives the Newton direction:

$$\nabla^2 f(x^{(k)})\, d = -\nabla f(x^{(k)}) \implies d^{(k)} = -\left[\nabla^2 f(x^{(k)})\right]^{-1} \nabla f(x^{(k)})$$

$$x^{(k+1)} = x^{(k)} + d^{(k)} = x^{(k)} - \left[\nabla^2 f(x^{(k)})\right]^{-1} \nabla f(x^{(k)})$$

The inverse Hessian $[\nabla^2 f(x^{(k)})]^{-1}$ acts on the gradient to reorient and rescale the search direction according to the local curvature of f: in directions where f curves steeply (large curvature), the step is short; in directions where f is nearly flat (small curvature), the step is long. This curvature-aware rescaling allows Newton's Method to bypass the zig-zagging behavior of Gradient Descent in ill-conditioned problems. For a quadratic objective, Newton's direction points exactly to the optimum in a single step.

Newton's Method converges quadratically near the optimum: the number of correct decimal digits roughly doubles at each iteration. However, this fast convergence is guaranteed only sufficiently close to the optimum; a line search is typically used when starting far away. The method also requires computing and inverting the full $n \times n$ Hessian at every step — expensive for large n.

1.4 In-Class Exercises

1.4.1 Exercise 1

An NHAI corridor study models total system travel time across two parallel links — the Mumbai–Pune Expressway (v_1) and old NH48 (v_2), with volumes in thousands of vehicles/hour — as:

$$T(v_1, v_2) = 3v_1^2 - 2v_1v_2 + 2v_2^2 - 14v_1 - 12v_2 + 50$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 3 iterations of Newton's Method by hand, starting from $(v_1, v_2) = (0, 0)$. Compare the result with the Gradient Descent trace from Lecture 13.

1.4.2 Exercise 2

Delhivery is minimizing operational cost at its Chennai cross-dock facility. The daily cost as a function of throughput λ (tonnes/day) and staffing level s (workers) is:

$$C(\lambda, s) = 2\lambda^2 - 4\lambda s + 3s^2 - 12\lambda - 6s + 50$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 3 iterations of Newton's Method by hand, starting from $(\lambda, s) = (0, 0)$. How many iterations are required to reach the optimum exactly, and why? Compare the result with the Gradient Descent trace from Lecture 13.

1.5 Take-Away Exercises

1.5.1 Exercise 1

BMTC is optimizing its demand-responsive feeder service in Bengaluru. The total daily operating cost depends on fleet size n (vehicles) and headway h (minutes). The cost model is:

$$C(n, h) = 2n^2 + 3h^2 + 2nh - 20n - 30h + 100$$

where the terms capture vehicle capital cost, passenger waiting cost, and a joint fleet-frequency interaction respectively.

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 1 iteration of Newton's Method by hand, starting from $(n, h) = (0, 0)$. Compare the result with the Gradient Descent trace from Lecture 13.

1.5.2 Exercise 2

An NHAI planning study models the full-lifecycle cost of a four-lane corridor as a function of traffic volume v (thousand veh/day) and maintenance frequency m (interventions/year):

$$F(v, m) = 3v^2 - 2vm + 2m^2 - 18v - 12m + 80$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Implement Newton's Method in Python using `scipy.optimize.minimize` with `method='Newton-CG'`, starting from $(v, m) = (0, 0)$. Report the optimal solution and number of iterations. Compare the result with the Gradient Descent trace from Lecture 13.

1.6 Further Reading

- Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer — Chapter 3 (line search methods), Chapter 7 (large-scale unconstrained optimization).
- Bazaraa, M.S., Sherali, H.D., and Shetty, C.M. (2006). *Nonlinear Programming: Theory and Algorithms* (3rd ed.). Wiley — Chapter 9 (Newton and quasi-Newton methods).
- SciPy documentation: `scipy.optimize.minimize` — docs.scipy.org
- BPR (1964). *Traffic Assignment Manual*. US Bureau of Public Roads, Washington DC.